

DataShare — Documentation technique

Projet réalisé dans le cadre du parcours Expert DevOps OpenClassrooms — Projet 3 : Pilotez le développement d'une solution informatique.

DataShare est une application de partage de fichiers. Un utilisateur dépose un fichier, choisit une durée de validité et, s'il le souhaite, un mot de passe, puis obtient un lien de téléchargement à transmettre. L'upload est aussi possible sans compte.

1. Architecture de l'application

DataShare suit une architecture classique en deux blocs séparés : une application web (front-end) qui parle à une API REST (back-end), elle-même connectée à une base de données relationnelle.

Vue d'ensemble

- **Front-end** : application React (SPA) servie par Vite. Elle s'exécute dans le navigateur de l'utilisateur et communique avec l'API en HTTP/JSON.
- **Back-end** : API REST développée avec Symfony. Elle expose les routes d'authentification et de gestion des fichiers, applique les règles de sécurité, et dialogue avec la base de données via Doctrine ORM.
- **Base de données** : PostgreSQL, qui stocke les comptes utilisateurs, les métadonnées des fichiers et leurs tags.
- **Stockage des fichiers** : les fichiers déposés sont enregistrés sur le disque du serveur (`backend/var/uploads`), sous le nom de leur token d'accès. Seules les métadonnées (nom d'origine, taille, type, expiration...) sont en base.

En développement, le front tourne sur `http://localhost:5173` et l'API sur `http://localhost:8000` . La base PostgreSQL est lancée via Docker Compose, sur le port `5433` .

Schéma de principe

Le schéma d'architecture (flux entre le navigateur, le front-end, l'API et la base de données) est disponible dans `livrable/DataShare_Architecture.drawio` .

Découpage du front-end

Le code front est organisé par fonctionnalité (pages, composants partagés, contexte d'authentification) :

- `src/pages` : les trois écrans principaux — Téléversement (upload), Téléchargement (page publique de récupération d'un fichier) et Mon espace (liste des fichiers de l'utilisateur connecté).
- `src/components` : composants UI réutilisables (boutons, champs, modale d'authentification, en-tête, pied de page...).

- `src/context/AuthContext.tsx` : gestion de la session (stockage du token JWT, état connecté/déconnecté).
- `src/lib/api.ts` : point d'entrée pour les appels à l'API.

La navigation est gérée par React Router : les routes publiques (`/` , `/televersement` , `/telechargement`) sont accessibles à tous, tandis que `/mon-espace` est protégée et redirige vers l'accueil si l'utilisateur n'est pas connecté.

Découpage du back-end

- `src/Controller/AuthController.php` : inscription, connexion (déléguée au firewall JWT) et gestion du compte (`/users/me`).
 - `src/Controller/FileController.php` : upload, téléchargement, listing, gestion des tags et suppression des fichiers.
 - `src/Entity` : les trois entités du modèle de données (`User` , `FileMetadata` , `Tag`), détaillées dans la section suivante.
 - `config/packages/security.yaml` : règles d'accès et configuration de l'authentification.
-

2. Choix technologiques justifiés

Symfony (back-end)

Alternatives écartées : Java, Node.

- Framework d'origine française : un atout commercial pour l'application, mais aussi un atout sur la popularité du framework en France et sur le vivier de développeurs qui le maîtrisent.
- Framework très structuré : il impose une normalisation poussée des projets et permet à un autre développeur de reprendre le projet rapidement.
- Architecture extensible, qui offre une bonne maintenabilité.
- Composants d'authentification et de sécurisation des données intégrés au framework.
- Architecture hexagonale native, bundles officiels pour l'authentification, l'ORM et les migrations, conventions strictes qui facilitent la maintenabilité.
- Migrations versionnées (`doctrine/doctrine-migrations-bundle`) et mappage objet-relationnel déclaratif, cohérents avec l'écosystème Symfony.
- Typage strict en PHP 8.4 (enums, fibers, readonly), écosystème Symfony mature, large bassin de développeurs PHP en France.

React (front-end)

Alternatives écartées : Vue, Angular.

- Open source, porté par un acteur majeur du secteur (Meta).
- Très populaire, ce qui offre un vivier de développeurs élargi.
- Interface fluide, facile à prendre en main.

- Autonomie totale vis-à-vis du composant serveur.
- Système de composants qui permet de réutiliser facilement les éléments déjà développés.
- TypeScript : détection des erreurs à la compilation, contrats d'interface explicites entre composants, autocomplétion fiable dans l'éditeur.
- JSX proche du HTML, donc accessible rapidement.
- Support à long terme assuré par Meta.

PostgreSQL (base de données)

Alternative écartée : MongoDB.

- Structure relationnelle, idéale pour gérer des utilisateurs et leurs relations avec leurs fichiers.
- Transactions ACID (atomicité, cohérence, isolation, durabilité) : garantit des données cohérentes.
- Permet des requêtes complexes (jointures, filtres).
- Authentification intégrée.

Stockage local des fichiers

Alternative écartée : S3.

- Très simple à mettre en place.
- Pas de coût externe.
- Rapide en local.
- Adapté à la taille de ce projet.

3. Modèle de données

Le modèle de données comporte trois tables, gérées par Doctrine ORM.

Table `app_user`

Représente un compte utilisateur.

Colonne	Type	Description
<code>id</code>	int (PK)	Identifiant auto-incrémenté
<code>email</code>	varchar(180), unique	Identifiant de connexion
<code>roles</code>	json	Rôles Symfony (toujours au moins <code>ROLE_USER</code>)
<code>password</code>	varchar(255)	Mot de passe haché
<code>created_at</code>	timestamp	Date de création du compte
<code>updated_at</code>	timestamp, nullable	Date de dernière modification

Table `file_metadata`

Représente un fichier déposé (les métadonnées ; le contenu binaire est sur le disque).

Colonne	Type	Description
<code>id</code>	int (PK)	Identifiant auto-incrémenté
<code>token</code>	varchar(64), unique	Identifiant public du fichier (32 caractères hexadécimaux), utilisé dans le lien de téléchargement et comme nom de fichier sur le disque
<code>original_name</code>	varchar(255)	Nom du fichier tel qu'envoyé par l'utilisateur
<code>storage_path</code>	varchar(500)	Chemin du fichier sur le disque
<code>size</code>	int	Taille en octets
<code>mime_type</code>	varchar(100)	Type MIME du fichier
<code>expires_at</code>	timestamp	Date d'expiration du lien (1 à 7 jours après le dépôt)
<code>password_hash</code>	varchar(255), nullable	Hash du mot de passe optionnel de protection du fichier
<code>created_at</code>	timestamp	Date de dépôt
<code>owner_id</code>	int (FK), nullable	Référence vers <code>app_user</code> ; <code>null</code> pour un dépôt anonyme

Table `tag`

Représente un tag associé à un fichier (un fichier peut avoir plusieurs tags, un même tag ne peut pas être présent deux fois sur un même fichier).

Colonne	Type	Description
<code>id</code>	int (PK)	Identifiant auto-incrémenté
<code>label</code>	varchar(30)	Libellé du tag
<code>file_id</code>	int (FK)	Référence vers <code>file_metadata</code> , suppression en cascade

Relations

- Un utilisateur (`app_user`) possède zéro ou plusieurs fichiers (`file_metadata`). La relation est optionnelle : un fichier peut ne pas avoir de propriétaire (upload anonyme).
- Un fichier (`file_metadata`) possède zéro ou plusieurs tags (`tag`). Si le fichier est supprimé, ses tags le sont automatiquement (suppression en cascade).

`app_user` (1) — (0..N) `file_metadata` (1) — (0..N) `tag`

4. Documentation d'API

L'API est documentée au format OpenAPI 3.0, dans `frontend/public/openapi.yaml` (consultable également via `frontend/dist/api-docs.html`). Une collection [Bruno](#) (`bruno/`) permet de tester chaque endpoint manuellement.

Authentification

Méthode	Route	Description	Accès
POST	<code>/auth/register</code>	Crée un compte (email + mot de passe, 8 caractères minimum)	Public
POST	<code>/auth/login</code>	Connexion, retourne un token JWT	Public
GET	<code>/users/me</code>	Récupère le profil du compte connecté	Authentifié
PUT	<code>/users/me</code>	Met à jour l'email et/ou le mot de passe	Authentifié
DELETE	<code>/users/me</code>	Supprime le compte	Authentifié

Fichiers

Méthode	Route	Description	Accès
POST	<code>/files</code>	Dépose un fichier (champ <code>file</code> , et en option <code>expires_in</code> , <code>password</code> , <code>tags</code>)	Public (anonyme ou connecté)
GET	<code>/files</code>	Liste les fichiers de l'utilisateur connecté	Authentifié
GET	<code>/files/{token}/info</code>	Renvoie les métadonnées d'un fichier (sans le contenu)	Public
GET	<code>/files/{token}</code>	Télécharge le fichier (paramètre <code>password</code> si protégé)	Public
PATCH	<code>/files/{id}/tags</code>	Remplace les tags d'un fichier	Authentifié, propriétaire uniquement
DELETE	<code>/files/{id}</code>	Supprime un fichier	Authentifié, propriétaire uniquement

Format des réponses

L'API répond en JSON. Les principaux objets retournés sont :

- **User** : `id`, `email`, `created_at`, `updated_at`.
- **FileMetadata** : `id`, `token`, `original_name`, `size`, `mime_type`, `expires_at`, `is_expired`, `tags`, `created_at`. Le mot de passe et le chemin de stockage ne sont jamais exposés.
- **UploadResponse** (réponse à `POST /files`) : `token`, `download_url`, `expires_at`.

- **Error** : `message` — message d'erreur lisible, retourné avec un code HTTP approprié (422, 401, 403, 404, 410, 413...).

Authentification dans les appels

Une fois connecté, le client doit envoyer le token JWT obtenu via `/auth/login` dans l'en-tête `Authorization: Bearer <token>` pour accéder aux routes protégées.

5. Sécurité et gestion des accès

Authentification

L'authentification repose sur des JWT (JSON Web Tokens), générés par `LexikJWTAuthenticationBundle`. Cette approche est sans état : le serveur ne conserve aucune session, toute l'information nécessaire est portée par le token signé envoyé à chaque requête. C'est une approche bien adaptée à une API REST consommée par une application front séparée.

Les mots de passe des comptes utilisateurs sont hachés avec l'algorithme `auto` de Symfony, qui choisit automatiquement l'algorithme le plus robuste disponible (bcrypt ou argon2) et permet de migrer en douceur si l'algorithme recommandé évolue.

Règles d'accès

Les règles d'accès sont définies explicitement dans `backend/config/packages/security.yaml` :

- **Routes publiques** : inscription (`/auth/register`), connexion (`/auth/login`), dépôt d'un fichier (`POST /files`, y compris en anonyme), consultation des métadonnées d'un fichier (`/files/{token}/info`) et téléchargement (`/files/{token}`).
- **Routes protégées** : tout ce qui touche au compte (`/users/...`) et à la gestion des fichiers d'un utilisateur (liste, tags, suppression) nécessite un JWT valide.

Pour les routes de téléchargement, le format du token est en plus contrôlé par une expression régulière (32 à 64 caractères hexadécimaux), ce qui limite les tentatives d'accès avec des valeurs invalides avant même d'atteindre le contrôleur.

Partage de fichiers

- Chaque fichier reçoit un **token aléatoire de 128 bits** (`bin2hex(random_bytes(16))`), qui sert à la fois de lien de partage et de nom de fichier sur le disque. Cela évite tout risque de collision ou de manipulation du chemin de fichier (path traversal) : le nom d'origine n'existe qu'en métadonnée, jamais sur le système de fichiers.
- Un fichier peut être protégé par un **mot de passe optionnel**, haché (jamais stocké en clair), avec une longueur minimale de 6 caractères. Pour aller plus loin, on pourrait exiger des caractères spéciaux ou une longueur minimale plus élevée, mais cela sort du périmètre du MVP.

- Chaque fichier a une **date d'expiration** (1 à 7 jours), au-delà de laquelle le téléchargement renvoie une erreur explicite.
- Les extensions de fichiers potentiellement dangereuses (exécutables, scripts...) sont refusées à l'upload, et la taille est limitée à 1 Go.

Réseau et secrets

- Les origines autorisées à appeler l'API sont contrôlées par CORS (NelmioCorsBundle), configurables via la variable d'environnement `CORS_ALLOW_ORIGIN`. Seuls les en-têtes `Content-Type` et `Authorization` sont acceptés.
- Les secrets (identifiants de la base de données, clés JWT) sont fournis par variables d'environnement et ne sont jamais commités dans le dépôt (`.env.local` est ignoré par Git).
- Les dépendances sont auditées régulièrement avec `npm audit` (front) et `composer audit` (back), voir la section suivante.

Limites connues, assumées pour le MVP

- Pas de limitation de débit (rate limiting) sur la connexion ou l'upload anonyme. À ajouter avant une mise en production réelle, par exemple avec `symfony/rate-limiter`.
- Pas d'analyse antivirus des fichiers déposés. C'est une évolution identifiée pour la suite.

6. Qualité, tests et maintenance

Stratégie de tests

Type de test	Outil	Périmètre
Tests unitaires / composants	Jest + React Testing Library	Pages et composants front (11 fichiers de tests)
Tests end-to-end	Cypress	Parcours utilisateurs complets (authentification, upload, téléchargement, navigation, espace personnel)
Tests d'API	Collection Bruno	Tous les endpoints back (auth, fichiers, tags, suppression)
Tests de performance	k6	Endpoints <code>/files/{token}/info</code> et <code>/auth/login</code> , montée en charge progressive

Critères d'acceptation

Avant de considérer une version comme livrable, on vérifie que :

1. Tous les tests unitaires passent (`npm test`).
2. Tous les scénarios Cypress passent en mode headless (`npm run cy:run`).
3. La couverture de code reste au-dessus de 70 % (couverture actuelle : environ 97-99 % selon les métriques).

4. Le lint ne remonte aucune erreur (`npm run lint`).
5. Le build de production réussit, vérification TypeScript incluse (`npm run build`).
6. Aucune régression sur les parcours critiques : connexion, dépôt, téléchargement protégé, expiration.
7. Tout nouveau développement est livré avec ses tests (et un scénario e2e si une nouvelle fonctionnalité utilisateur est ajoutée).

Performance

Des tests de charge avec k6 valident que l'API tient des temps de réponse raisonnables (objectif : 95 % des requêtes en moins de 200 ms, 99 % en moins de 500 ms, moins de 1 % d'erreurs). Côté front, des budgets sont fixés sur la taille du bundle et les indicateurs Lighthouse (FCP, LCP). Le détail et les derniers résultats sont dans `PERF.md` .

Maintenance des dépendances

Les dépendances front (npm) et back (Composer) sont mises à jour suivant un même processus : création d'une branche dédiée, mise à jour, relance des audits de sécurité, validation par la suite de tests, puis pull request. La fréquence dépend du type de mise à jour :

- **Correctifs de sécurité** : sous 48 heures.
- **Mises à jour mineures/patches** : mensuelles.
- **Mises à jour majeures** (React, Symfony, PostgreSQL...) : trimestrielles, une à la fois, avec lecture du changelog.
- **Audit de sécurité seul** : au moins une fois par mois, et à chaque pull request touchant aux dépendances.

Le détail des risques identifiés (changements cassants, désynchronisation des lockfiles, migrations de schéma...) et des parades associées est documenté dans `MAINTENANCE.md` .

7. Processus d'installation et d'exécution

Prérequis

- Docker et Docker Compose (pour la base de données)
- PHP $\geq$ 8.4 et Composer
- Node.js $\geq$ 20 et npm
- Optionnel : Symfony CLI

Installation rapide

Un script unique automatise toute l'installation :

```
./setup.sh
```

Ce script démarre la base PostgreSQL via Docker, installe les dépendances back et front, génère les clés JWT, exécute les migrations et charge des données de démonstration. Il ne lance pas les serveurs applicatifs.

Installation manuelle

Base de données

```
cd backend
docker compose up -d database
```

Back-end (Symfony)

```
cd backend
composer install
php bin/console lexik:jwt:generate-keypair
php bin/console doctrine:migrations:migrate
php bin/console doctrine:fixtures:load # optionnel, données de démo
symfony server:start # ou : php -S 127.0.0.1:8000 -t public
```

L'API est alors disponible sur `http://localhost:8000`.

Front-end (React)

```
cd frontend
npm install
npm run dev
```

L'application est disponible sur `http://localhost:5173`.

Variables d'environnement

Variable	Description
<code>DATABASE_URL</code>	DSN de connexion PostgreSQL
<code>CORS_ALLOW_ORIGIN</code>	Origines autorisées pour les appels API
<code>JWT_SECRET_KEY</code> / <code>JWT_PUBLIC_KEY</code>	Chemins des clés JWT
<code>JWT_PASSPHRASE</code>	Passphrase de la clé privée JWT

Ces variables se configurent dans `backend/.env.local` (non versionné). Les paramètres de la base PostgreSQL (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_VERSION`) se trouvent dans `backend/compose.yaml` / `compose.override.yaml`.

Utilisation

Une fois les trois éléments démarrés (base, back, front) :

1. Créer un compte via la page d'accueil (inscription).
 2. Se connecter.
 3. Déposer un fichier, choisir sa durée de validité et, si besoin, un mot de passe.
 4. Récupérer le lien de téléchargement et le partager.
 5. Retrouver, taguer ou supprimer ses fichiers depuis « Mon espace ».
-

8. Utilisation de l'IA dans le développement

L'IA (Claude Code) a été utilisée tout au long du projet comme un outil d'aide au développement, en complément du travail de conception et de décision qui reste humain.

Génération et complément de code

Claude Code a été sollicité pour écrire ou compléter des composants front (pages, formulaires, composants partagés), des endpoints back (contrôleurs, entités Doctrine) et leurs tests associés (unitaires Jest/RTL, scénarios Cypress, tests d'API). Cela a permis d'aller plus vite sur l'écriture du code répétitif (CRUD, formulaires, mapping de données) et de générer un socle de tests dès l'ajout d'une fonctionnalité, ce qui a contribué à la couverture de tests élevée du projet (proche de 97-99 % côté front).

Dans tous les cas, le code généré a été relu, ajusté et testé avant d'être intégré : l'IA a servi de point de départ et d'accélérateur, pas de validation finale.

Documentation

Une partie de la documentation du projet (le présent document, ainsi que `SECURITY.md`, `TESTING.md`, `MAINTENANCE.md` et `PERF.md`) a été rédigée avec l'aide de l'IA, à partir du code et des décisions déjà prises sur le projet. L'IA a notamment aidé à :

- structurer les documents et reformuler les choix techniques de manière claire ;
- décrire les scénarios de test et les critères d'acceptation à partir des suites de tests existantes ;
- formaliser les procédures de mise à jour des dépendances et les bonnes pratiques de sécurité déjà appliquées dans le code.

Le contenu reste basé sur l'état réel du projet (code, configuration, résultats de tests) : l'IA a servi à gagner du temps sur la mise en forme et la rédaction, pas à inventer des fonctionnalités ou des résultats.